



COMSATS University Islamabad,
Abbottabad Campus

SOFTWARE DESIGN DESCRIPTION
(SDD DOCUMENT)

for
Depression Alert AI

Version 1.0

By

Muhammad Salahudin Khan

CIIT/SP23-BCS-135/ATD

Muhammad Azan

CIIT/SP23-BCS-116/ATD

Supervisor

Ms. Sara Shafique

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

Revision History	3
Application Evaluation History	3
1. Introduction	4
2. Design Methodology and Software Process Model	4
3. System Overview	5
3.1 Architectural Design	5
3.2 Process Flow/Representation	6
4. Design Models [along with descriptions]	8
4.1 Class Diagram	8
4.2 Sequence Diagram	8
4.4 Data Flow Diagram (Level 1)	10
5. Data Design	10
5.1 Data Dictionary	10
6. Algorithm & Implementation	13
6.1 Data Collection Module — validateAndStorePost()	13
6.2 NLP Processing Module — preprocessText()	13
6.3 Depression Detection Module — computeRiskScore()	13
6.4 Alert System Module — evaluateAndGenerateAlert()	14
6.5 Behavioral Analysis Module — updateBehavioralPattern()	14
7. Software Requirements Traceability Matrix	14
8. Human Interface Design	15
8.1 Screen Images	15
8.1.1 Dashboard	16
8.1.2 Daily Log	16
8.1.3 Sentiment Alert	18
8.1.4 Crisis Support & Resources	19
8.2 Screen Objects and Actions	20
9. Appendix I	21

Revision History

Name	Date	Reason for Changes	Version
Muhammad Salahudin Khan, Muhammad Azan	13-Jul-2026	Initial draft of Software Design Description	1.0

Application Evaluation History

Comments (by Committee)	Action Taken

Supervised by
Ms. Sara Shafique

Signature: _____

1. Introduction

This Software Design Description (SDD) document presents the detailed technical design of DepressionAlert AI, an intelligent web-based system developed to identify early signs of depression by analyzing user-provided social-media text using Natural Language Processing (NLP) and Machine Learning (ML) techniques. This document builds upon the DepressionAlert AI Project Proposal and translates the requirements and scope defined therein into a concrete architectural, data, and interface design that will guide implementation.

The scope of the project covered in this design consists of six core modules: (1) the Data Collection Module, which allows users to submit social-media posts either by pasting text directly or by uploading CSV files after providing informed consent; (2) the NLP Processing Module, which cleans, tokenizes, and extracts sentiment and linguistic features from the submitted text; (3) the Depression Detection Module, which applies a trained machine-learning classifier to the extracted features to compute a depression risk score; (4) the Behavioral Analysis Module, which studies patterns of language use (e.g., first-person pronoun density, absolutist language, negative-emotion word frequency) across a user's submissions over time; (5) the Alert System Module, which raises notifications whenever a risk score crosses a defined threshold; and (6) the Dashboard Module, which presents analysis history, sentiment trends, and alerts to the user in an organized visual interface.

The system is strictly permission-based: no data is collected automatically, and all monitoring begins only after explicit user consent. DepressionAlert AI is intended to provide supportive early-awareness insight and is not a diagnostic or clinical tool; all outputs are advisory and are meant to complement, not replace, professional mental-health evaluation.

This document is intended for the development team, the project supervisor, and the FYP evaluation committee, and it serves as the primary reference for implementation, verification, and future maintenance of the system.

2. Design Methodology and Software Process Model

DepressionAlert AI is designed using the Object-Oriented (OO) design paradigm. An object-oriented approach was chosen because the system is naturally composed of distinct, self-contained entities — such as User, Post, ProcessedText, AnalysisResult, and Alert — each of which encapsulates its own data and behavior. Object orientation allows these entities to be modeled directly as classes with clear responsibilities, supports reuse of common behavior through inheritance (for example, a shared base Notification class extended by in-app and email alert types), and makes the system easier to extend as new analysis features (e.g., additional linguistic markers or new ML models) are added in the future without disturbing existing modules.

The system follows a layered, service-oriented internal structure in which the presentation layer (React front end), the application/business logic layer (Node.js/Express APIs and Python-based ML services), and the data layer (database and file storage) are kept separate. This separation of concerns improves maintainability, allows independent testing of each layer, and supports the modular decomposition described in Section 3.

For the overall software process, the team is following the Agile Software Development methodology, using short, iterative development cycles (sprints). Agile was selected because the project requirements — particularly around NLP feature selection and ML model tuning — are expected to

evolve as the team experiments with different techniques and datasets. Agile's iterative nature allows the team to build, test, and refine one module at a time (e.g., completing the Data Collection Module before moving to NLP Processing), incorporate supervisor feedback continuously, and adjust the risk-scoring approach based on evaluation results without disrupting the entire project timeline. Regular sprint reviews are held with the project supervisor, and GitHub is used for version control, issue tracking, and team collaboration, as reflected in the project Gantt chart.

3. System Overview

DepressionAlert AI is a web-based Artificial Intelligence application that enables a user to submit social-media text (via manual paste or CSV upload) and receive an automated assessment of possible depression-related linguistic and emotional patterns. At a general level, the system accepts raw text, transforms it through a natural-language processing pipeline into a structured feature representation, passes those features through a trained classification model to produce a numeric risk score, and then presents the result to the user through a dashboard, raising an alert if the score exceeds a configured threshold.

The system is built as a three-tier web application. The client tier is a React single-page application that provides the dashboard, daily log, sentiment alert, and crisis-support screens. The server tier consists of a Node.js/Express API layer that handles authentication, request routing, and orchestration, together with a Python-based analysis service (using scikit-learn, NLTK, and spaCy) that performs the actual NLP processing and ML inference. The data tier stores user accounts, submitted posts, processed text, analysis results, and alerts in a persistent database. This layered separation allows the AI/ML component to be developed, trained, and updated independently of the web application logic.

3.1 Architectural Design

The architecture of DepressionAlert AI is organized as a layered (multi-tiered) client-server system, decomposed into the six functional modules identified in Section 1. The Presentation Layer (React Dashboard, Daily Log, Sentiment Alert, and Crisis Support screens) communicates with the Application Layer over REST APIs. The Application Layer coordinates the Data Collection Module, which validates and stores incoming posts; the NLP Processing Module, which cleans and vectorizes text; the Depression Detection Module, which invokes the trained ML model; the Behavioral Analysis Module, which aggregates linguistic markers across time; and the Alert System Module, which evaluates thresholds and creates notifications. All modules read from and write to the Data Layer, which persists Users, Posts, ProcessedText, AnalysisResults, and Alerts. This decomposition keeps each module responsible for a single concern, allows modules to be tested in isolation, and allows the ML model used by the Depression Detection Module to be retrained or replaced without requiring changes to the other modules.

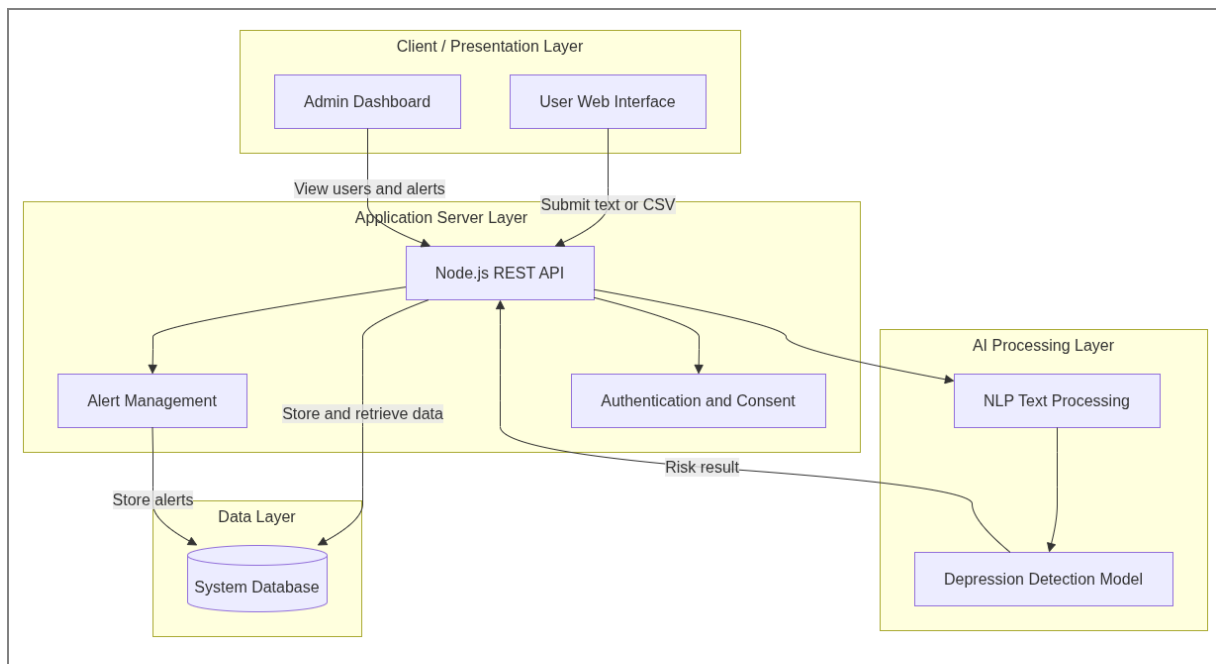


Figure 3.1: High-Level Layered / Client-Server Architecture Diagram of DepressionAlert AI

3.2 Process Flow/Representation

The major process of the system is the end-to-end text-analysis workflow, which begins the moment a user submits text and ends when a result is displayed (and, where applicable, an alert is raised). This is the only process complex enough to warrant an activity diagram, since it involves conditional branching (CSV vs. manual input, threshold checks), parallel-capable steps (feature extraction), and multiple possible outcomes (normal result vs. alert).

At a high level, the flow proceeds as follows: the user provides consent and submits a post (via paste or CSV upload); the Data Collection Module validates the submission and stores it; the NLP Processing Module cleans the text, removes stop words and irrelevant symbols, tokenizes the content, and extracts sentiment and linguistic features; the Depression Detection Module feeds these features into the trained classifier to compute a risk score; the Behavioral Analysis Module updates the user's historical pattern data; the system checks whether the risk score exceeds the alert threshold — if it does, the Alert System Module generates a notification, otherwise the result is simply logged; and finally the Dashboard Module updates to reflect the new analysis result, risk trend, and (if applicable) alert.

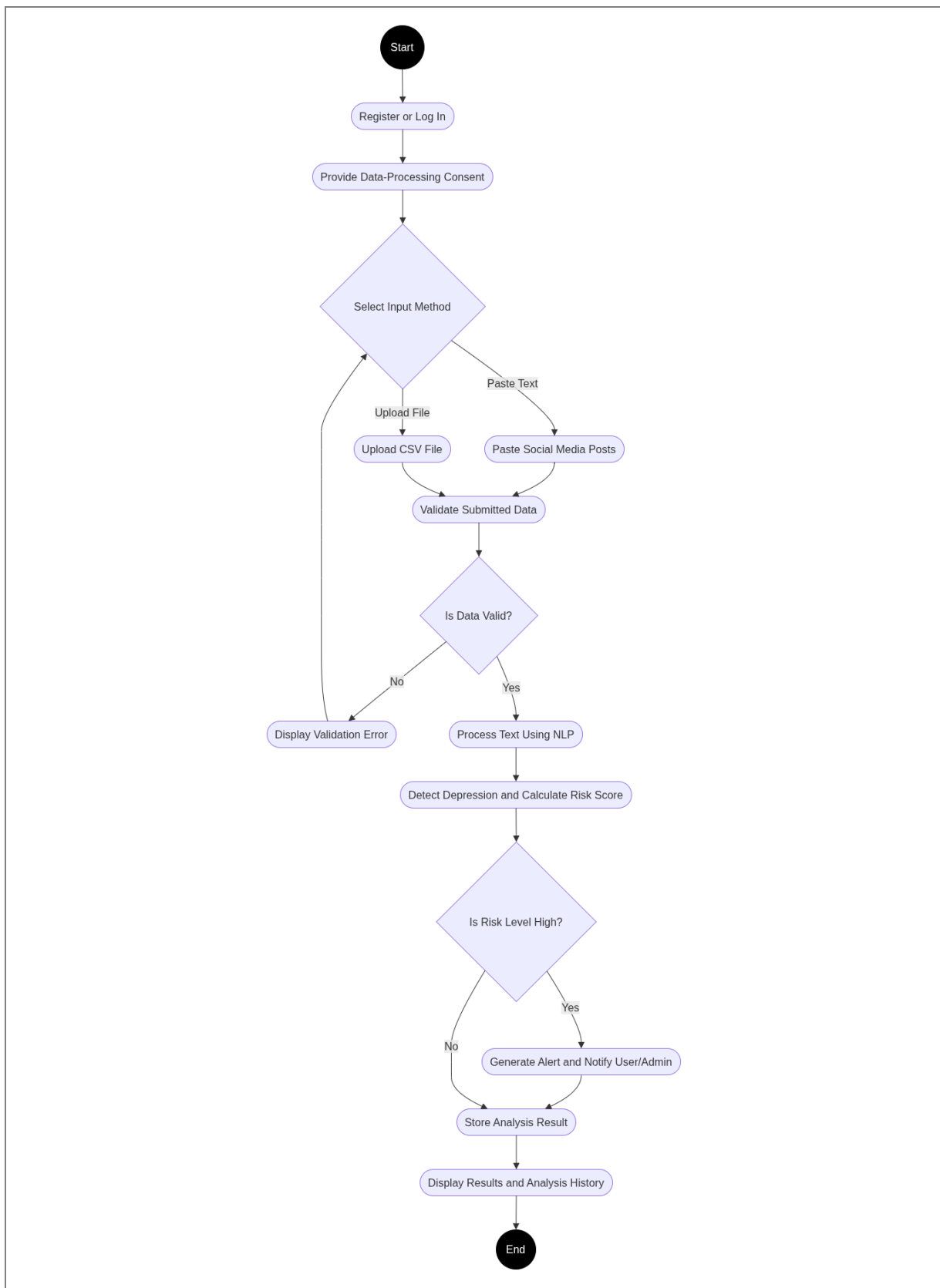


Figure 3.2: Activity Diagram — Text Submission to Risk-Score and Alert Generation Workflow

4. Design Models [along with descriptions]

Because DepressionAlert AI follows an object-oriented design approach, the applicable design models are the Class Diagram, Sequence Diagram, and State Transition Diagram. A Data Flow Diagram (DFD) is additionally included to clarify how data moves between the processing modules and the data stores, since this view is useful for an analysis-pipeline system such as this one.

4.1 Class Diagram

The class diagram captures the core entities of the system and their relationships. The central classes are User, Post, ProcessedText, AnalysisResult, and Alert. A User has a one-to-many association with Post (a user submits many posts). Each Post is associated with exactly one ProcessedText, produced by the NLP Processing Module, which in turn is associated with one AnalysisResult produced by the Depression Detection Module. An AnalysisResult may generate zero or one Alert, depending on whether its risk score crosses the threshold. A base Notification class is generalized into InAppAlert and (optionally) EmailAlert subclasses, illustrating inheritance. Aggregation is used to represent the Dashboard's composition of multiple AnalysisResult and Alert summaries without owning their lifecycle, since these objects continue to exist independently of any single dashboard view.

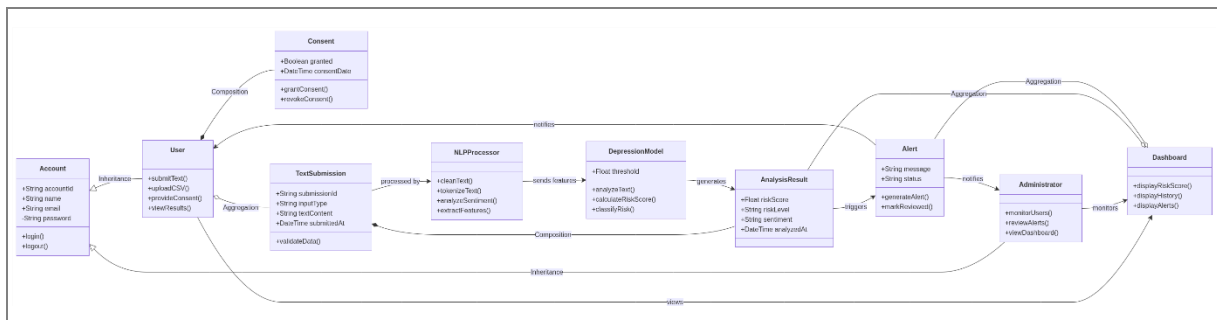


Figure 3.3: Class Diagram — User, Post, ProcessedText, AnalysisResult, Alert, and Notification hierarchy

4.2 Sequence Diagram

The sequence diagram illustrates the interaction between the User, the Dashboard (UI), the Data Collection Module, the NLP Processing Module, the Depression Detection Module, and the Alert System Module during a single post-submission cycle. The User submits a post through the UI; the UI calls the Data Collection Module to validate and persist it; the Data Collection Module invokes the NLP Processing Module to clean and extract features; the NLP Processing Module returns the feature vector to the Depression Detection Module, which computes and returns a risk score; the orchestrating service checks the score against the threshold and, if exceeded, calls the Alert System Module to create and return an Alert object; finally, the UI is updated with the new AnalysisResult (and Alert, if any).

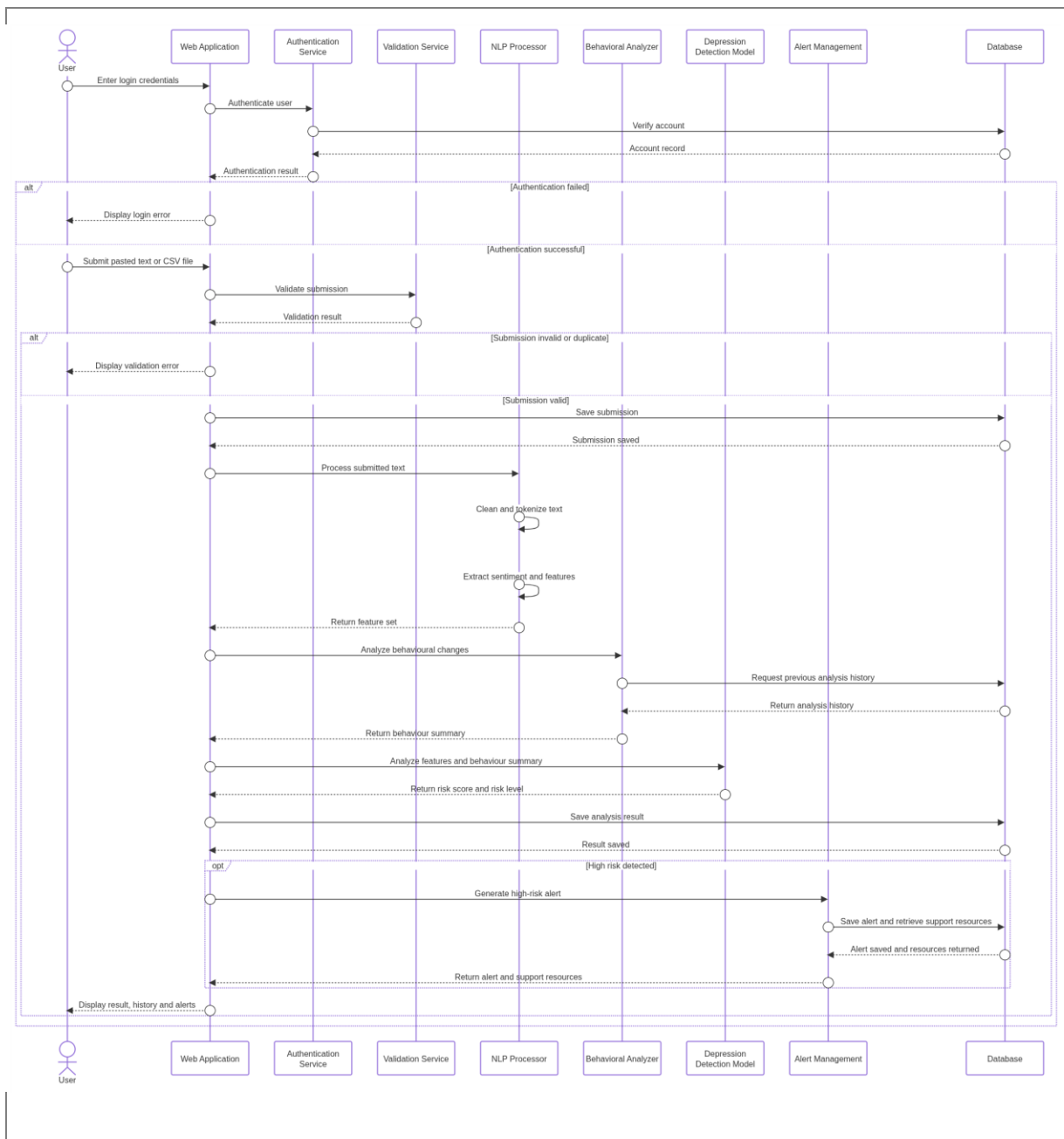


Figure 3.4: Sequence Diagram — Post Submission, Analysis, and Alert Generation

4.4 Data Flow Diagram (Level 1)

The Level-1 DFD shows the flow of data between the external entity (User), the six processing modules, and the data stores. The User (source) submits post data to Process 1 – Collect Data, which writes validated posts to D1: Post Store. Process 2 – Preprocess Text reads from D1, cleans and tokenizes the text, and writes results to D2: Processed Text Store. Process 3 – Detect Depression reads D2, applies the ML model, and writes results to D3: Analysis Result Store. Process 4 – Analyze Behavior reads D3 across multiple records to update D4: Behavioral Pattern Store. Process 5 – Generate Alerts reads D3 and D4, and where the threshold is exceeded, writes to D5: Alert Store. Process 6 – Update Dashboard reads D3, D4, and D5, and returns the aggregated view (sink) back to the User.

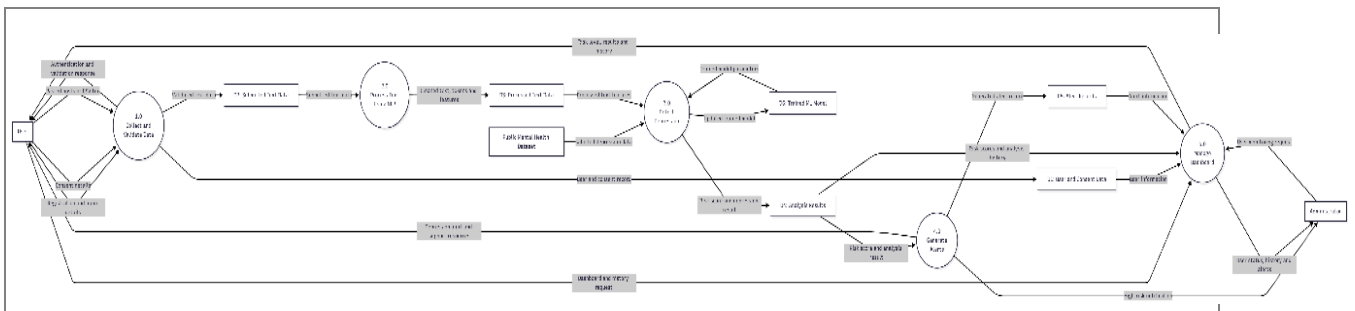


Figure 3.5: Level-1 Data Flow Diagram — DepressionAlert AI Processing Pipeline

5. Data Design

The information domain of DepressionAlert AI is transformed into a small set of related data structures that mirror the six functional modules. Raw user submissions (Post) are transformed by the NLP Processing Module into ProcessedText, which captures cleaned tokens and a preliminary sentiment score. ProcessedText is transformed by the Depression Detection Module into an AnalysisResult, which holds the numeric risk score and categorical risk level. Across multiple AnalysisResult records for the same user, the Behavioral Analysis Module derives a BehavioralPattern summary capturing linguistic-marker trends (e.g., increased first-person pronoun usage, absolutist language, negative-emotion word frequency) used for the Sentiment Volatility timeline on the dashboard. When an AnalysisResult's risk score exceeds the configured threshold, an Alert record is created and linked back to that result.

All entities are stored in a relational database (e.g., PostgreSQL/MySQL) accessed through the Node.js/Express application layer. Raw CSV files uploaded by users are temporarily held in secure server-side storage during validation and are discarded once their contents have been persisted as individual Post records, in order to minimize the retention of unnecessary raw files. User credentials are stored using a salted password hash rather than plain text, and all analysis data is associated with a user_id so that access can be restricted to the owning user and explicitly authorized viewers only.

5.1 Data Dictionary

The table below lists, in alphabetical order, the major system entities and their attributes, along with their data types and descriptions.

Entity / Attribute	Type	Description
Alert	Object	Represents a notification generated when a user's AnalysisResult risk score exceeds the configured threshold; displayed on the Dashboard and Sentiment Alert screen.
Alert.alert_id	Integer (PK)	Unique identifier for an alert record.
Alert.analysis_result_id	Integer (FK)	Reference to the AnalysisResult that triggered the alert.
Alert.created_at	DateTime	Timestamp when the alert was generated.
Alert.message	String	Human-readable alert message shown to the user (e.g., high concentration of negative sentiment detected).
Alert.status	Enum (New, Viewed, Dismissed)	Current state of the alert as acted on by the user.
AnalysisResult	Object	Represents the output of the Depression Detection Module for a single processed post.
AnalysisResult.analysis_id	Integer (PK)	Unique identifier for an analysis result.
AnalysisResult.processed_text_id	Integer (FK)	Reference to the ProcessedText that was analyzed.
AnalysisResult.risk_score	Float (0-100)	Numeric depression-risk score produced by the ML classifier.
AnalysisResult.risk_level	Enum (Low, Moderate, High)	Categorical risk band derived from risk_score.
AnalysisResult.model_version	String	Identifier of the ML model version used to produce this result, for traceability.
AnalysisResult.analyzed_at	DateTime	Timestamp of when the analysis was performed.
BehavioralPattern	Object	Aggregated linguistic-marker statistics for a user across multiple posts over time (e.g., first-person pronoun density, absolutist language, negative-emotion word frequency).
BehavioralPattern.pattern_id	Integer (PK)	Unique identifier for a behavioral pattern record.
BehavioralPattern.user_id	Integer (FK)	Reference to the associated User.
BehavioralPattern.window_start /	DateTime	Time window over which the

Entity / Attribute	Type	Description
window_end		aggregated pattern was computed.
Post	Object	Represents a single piece of user-submitted text (manually pasted or extracted from a CSV row).
Post.post_id	Integer (PK)	Unique identifier for a post.
Post.user_id	Integer (FK)	Reference to the User who submitted the post.
Post.content	Text	Raw text content of the submitted post.
Post.source	Enum (Manual, CSV)	Indicates how the post was submitted.
Post.submitted_at	DateTime	Timestamp when the post was submitted.
ProcessedText	Object	Represents the cleaned and feature-extracted version of a Post produced by the NLP Processing Module.
ProcessedText.processed_id	Integer (PK)	Unique identifier for a processed-text record.
ProcessedText.post_id	Integer (FK)	Reference to the source Post.
ProcessedText.cleaned_text	Text	Text after stop-word removal and symbol cleaning.
ProcessedText.tokens	Array<String>	Tokenized representation of the cleaned text.
ProcessedText.sentiment_score	Float (-1 to 1)	Sentiment polarity score produced by sentiment analysis.
User	Object	Represents a registered individual who uses the system, either as a self-monitoring user or an authorized viewer.
User.user_id	Integer (PK)	Unique identifier for a user account.
User.name	String	Full name of the user.
User.email	String	Email address used for login and notifications.
User.password_hash	String	Securely hashed password credential.
User.consent_given	Boolean	Flag indicating the user has explicitly consented to text analysis.
User.role	Enum (Standard, Authorized Viewer)	Access level of the user within the system.
User.created_at	DateTime	Timestamp when the account was

Entity / Attribute	Type	Description
		created.

6. Algorithm & Implementation

This section summarizes, in procedural description language (PDL)/pseudocode, the core algorithm implemented by each major module's primary function.

6.1 Data Collection Module — `validateAndStorePost()`

```

FUNCTION validateAndStorePost(userId, rawInput, sourceType):
  IF userConsent(userId) == FALSE THEN
    RETURN Error("Consent required")
  IF sourceType == CSV THEN
    rows = parseCSV(rawInput)
    FOR EACH row IN rows:
      IF row.content IS empty OR duplicate THEN CONTINUE
      post = new Post(userId, row.content, CSV, now())
      SAVE post TO PostStore
  ELSE
    IF rawInput IS empty THEN RETURN Error("Empty submission")
    post = new Post(userId, rawInput, MANUAL, now())
    SAVE post TO PostStore
  RETURN success

```

6.2 NLP Processing Module — `preprocessText()`

```

FUNCTION preprocessText(post):
  text = removeSymbolsAndUrls(post.content)
  text = toLowerCase(text)
  tokens = tokenize(text)
  tokens = removeStopWords(tokens)
  sentimentScore = computeSentiment(tokens)
  featureVector = extractFeatures(tokens, sentimentScore)
  processed = new ProcessedText(post.id, text, tokens, sentimentScore)
  SAVE processed TO ProcessedTextStore
  RETURN featureVector

```

6.3 Depression Detection Module — `computeRiskScore()`

```

FUNCTION computeRiskScore(featureVector, processedTextId):
  model = loadTrainedModel(currentModelVersion)
  rawScore = model.predict_proba(featureVector)
  riskScore = rawScore * 100
  IF riskScore >= 70 THEN riskLevel = "High"
  ELSE IF riskScore >= 40 THEN riskLevel = "Moderate"
  ELSE riskLevel = "Low"
  result = new AnalysisResult(processedTextId, riskScore, riskLevel, modelVersion, now())
  SAVE result TO AnalysisResultStore
  RETURN result

```

6.4 Alert System Module — evaluateAndGenerateAlert()

```

FUNCTION evaluateAndGenerateAlert(analysisResult, userId):
  IF analysisResult.riskLevel == "High" THEN
    message = buildAlertMessage(analysisResult)
    alert = new Alert(analysisResult.id, message, "New", now())
    SAVE alert TO AlertStore
    notifyUser(userId, alert)
    RETURN alert
  ELSE
    RETURN null

```

6.5 Behavioral Analysis Module — updateBehavioralPattern()

```

FUNCTION updateBehavioralPattern(userId, processedText):
  pattern = loadOrCreatePattern(userId, currentWindow())
  pattern.firstPersonDensity = updateAverage(pattern.firstPersonDensity,
countFirstPersonPronouns(processedText.tokens))
  pattern.absolutistLanguage = updateAverage(pattern.absolutistLanguage,
countAbsolutistWords(processedText.tokens))
  pattern.negativeEmotionWords = updateAverage(pattern.negativeEmotionWords,
countNegativeWords(processedText.tokens))
  SAVE pattern TO BehavioralPatternStore
  RETURN pattern

```

7. Software Requirements Traceability Matrix

The table below summarizes how each functional requirement (FR) of DepressionAlert AI has been addressed by a specific design component, mapping each requirement to the relevant diagram, class or process, and implementing function(s).

Table 1: Requirements Traceability Matrix

Req. Number	Ref. Item	Design Component	Component Items
FR01	Class Diagram	User	registerConsent(), authenticate()
FR02	Class / DFD	Post / Process 1	validateAndStorePost()
FR03	Class / DFD	Post / Process 1	validateAndStorePost() (manual path)
FR04	Class / DFD	ProcessedText / Process 2	preprocessText(), computeSentiment()
FR05	Class / DFD	AnalysisResult / Process 3	computeRiskScore()
FR06	Class / DFD	BehavioralPattern / Process 4	updateBehavioralPattern()
FR07	Class / DFD	Alert / Process 5	evaluateAndGenerateAlert(), notifyUser()
FR08	Class / Sequence	Dashboard	loadAnalysisHistory(), renderDashboard()

8. Human Interface Design

From the user's perspective, DepressionAlert AI is accessed through a web dashboard that requires the user to log in and grant explicit consent before any text analysis begins. Once inside the application, the user can submit posts, review sentiment and risk trends, respond to alerts, and access crisis-support resources. The interface emphasizes clarity and calm visual design given the sensitive nature of the content, using a left-hand navigation sidebar (Dashboard, Daily Log, Privacy, Crisis Support) and a card-based main content area. Feedback to the user is provided through color-coded risk indicators (e.g., green for low risk, amber/red for elevated risk), progress and trend charts, and clearly worded, non-alarming alert banners with actionable next steps (e.g., "Take a 30-min break" or "Dismiss").

8.1 Screen Images

The following screens represent the primary views of the system, based on the approved project mockups.

8.1.1 Dashboard

The Dashboard is the landing screen after login. It displays the user's current Digital Sentiment Score, a 7-day historical impact chart correlating usage time with mood score, a mood-impact-by-platform breakdown, a set of preventive-action shortcuts, and a live social-media sentiment monitor feed showing recent AI-detected pattern observations.

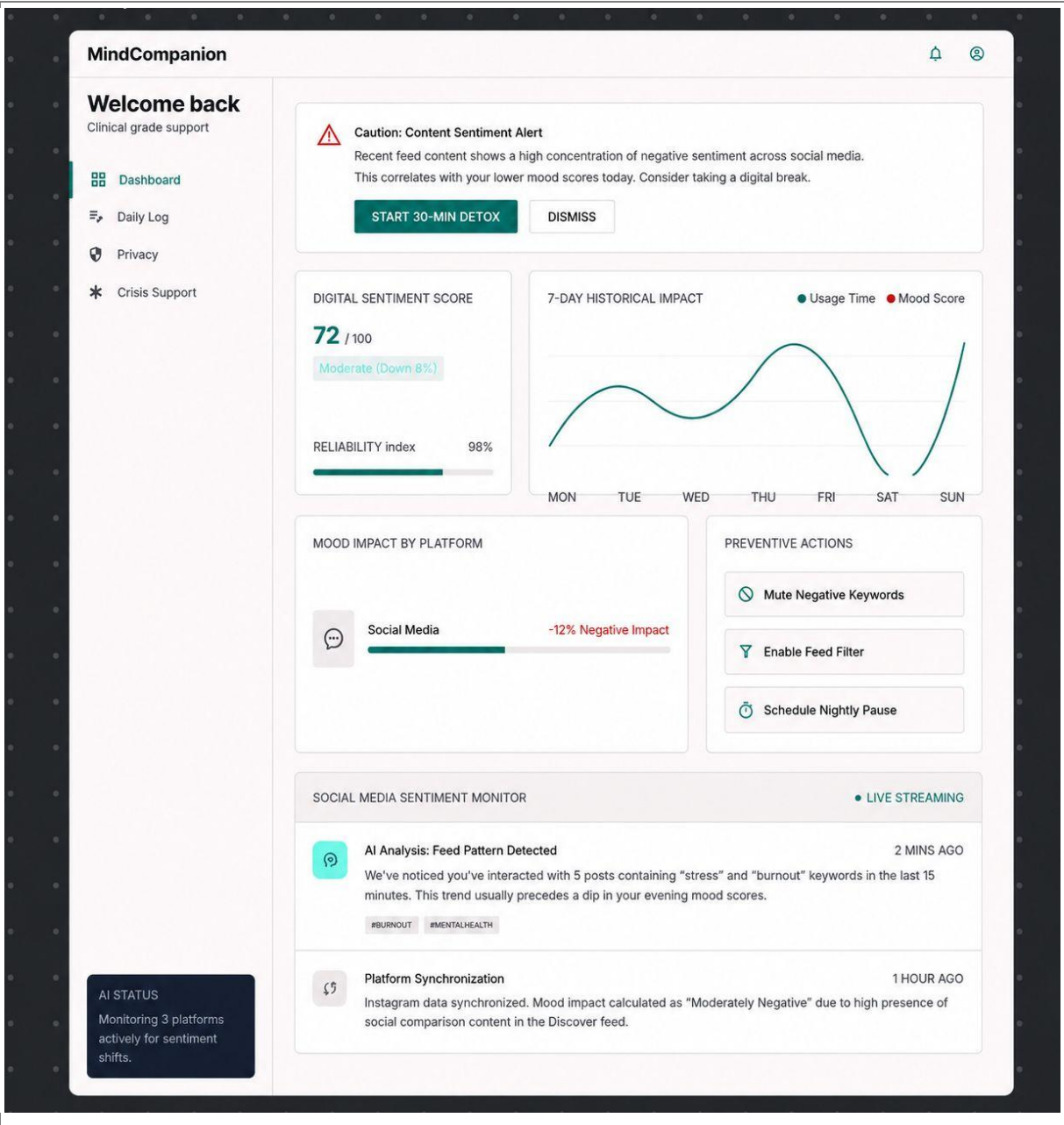


Figure 8.1.1: Screen Image — Dashboard (Sentiment Score, Trend Chart, Preventive Actions)

8.1.2 Daily Log

The Daily Log screen presents a Linguistic Marker Analysis panel (first-person pronoun density, absolutist language usage, negative-emotion word levels), a Sentiment Volatility timeline chart highlighting days with elevated risk, and a Post & Comment Deep Dive panel that overlays AI-detected linguistic indicators directly on the user's own recent posts and comments.

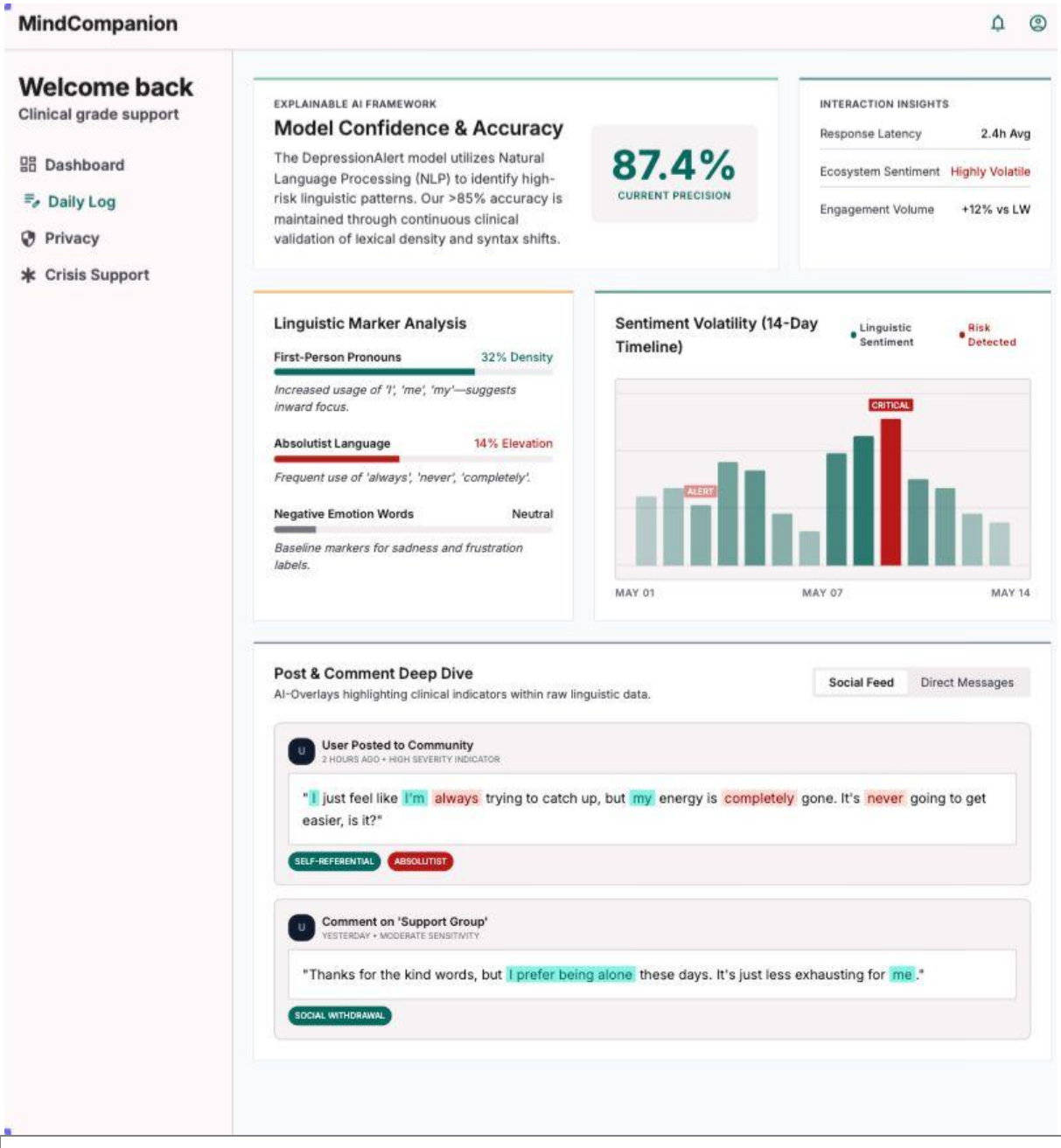


Figure 8.1.2: Screen Image — Daily Log (Linguistic Markers, Sentiment Volatility, Deep Dive)

8.1.3 Sentiment Alert

The Sentiment Alert screen is a modal notification shown when the system detects a high concentration of negative sentiment in the user's recent feed. It presents a brief explanation of the detected pattern and offers two direct actions: starting a short guided break, or dismissing the alert.

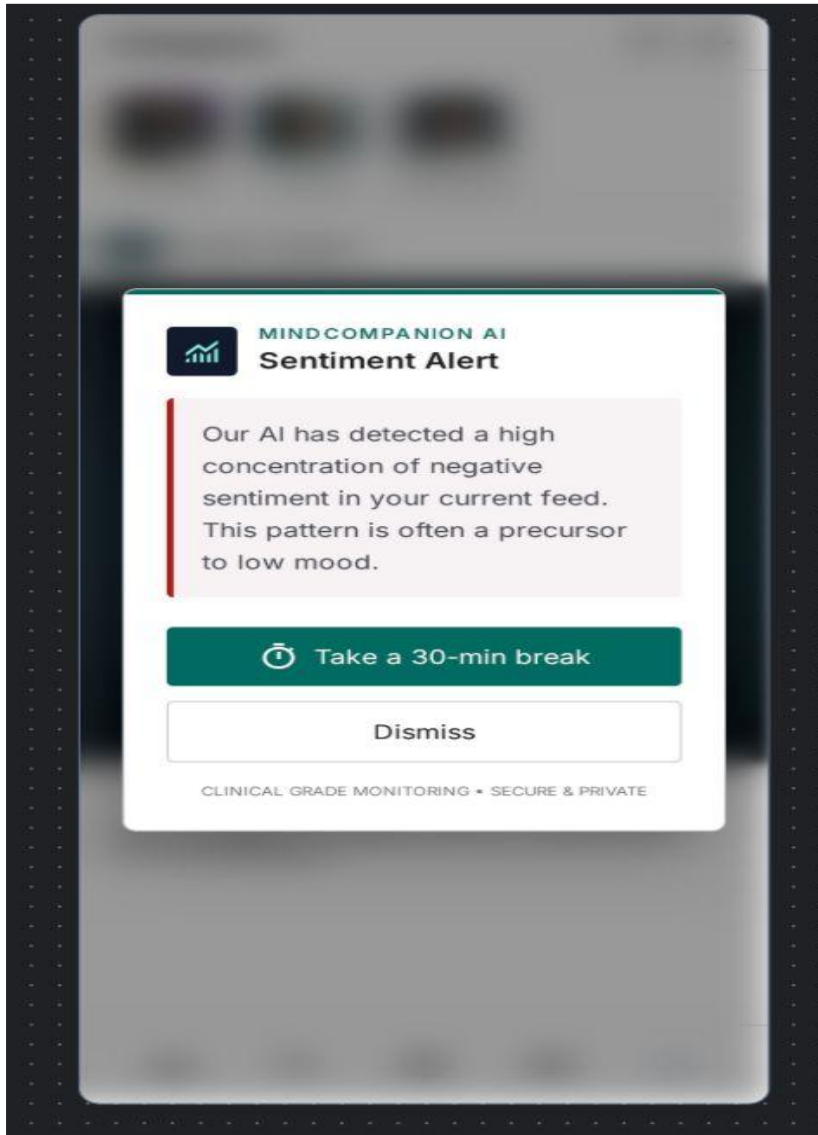


Figure 8.1.3: Screen Image — Sentiment Alert Modal

8.1.4 Crisis Support & Resources

The Crisis Support & Resources screen provides the user with a toolkit of coping resources (breathing exercises, guided meditation, mood-boosting audio) and a learning library of articles related to sentiment tracking and AI in mental health, giving users constructive next steps beyond simply viewing their risk data.

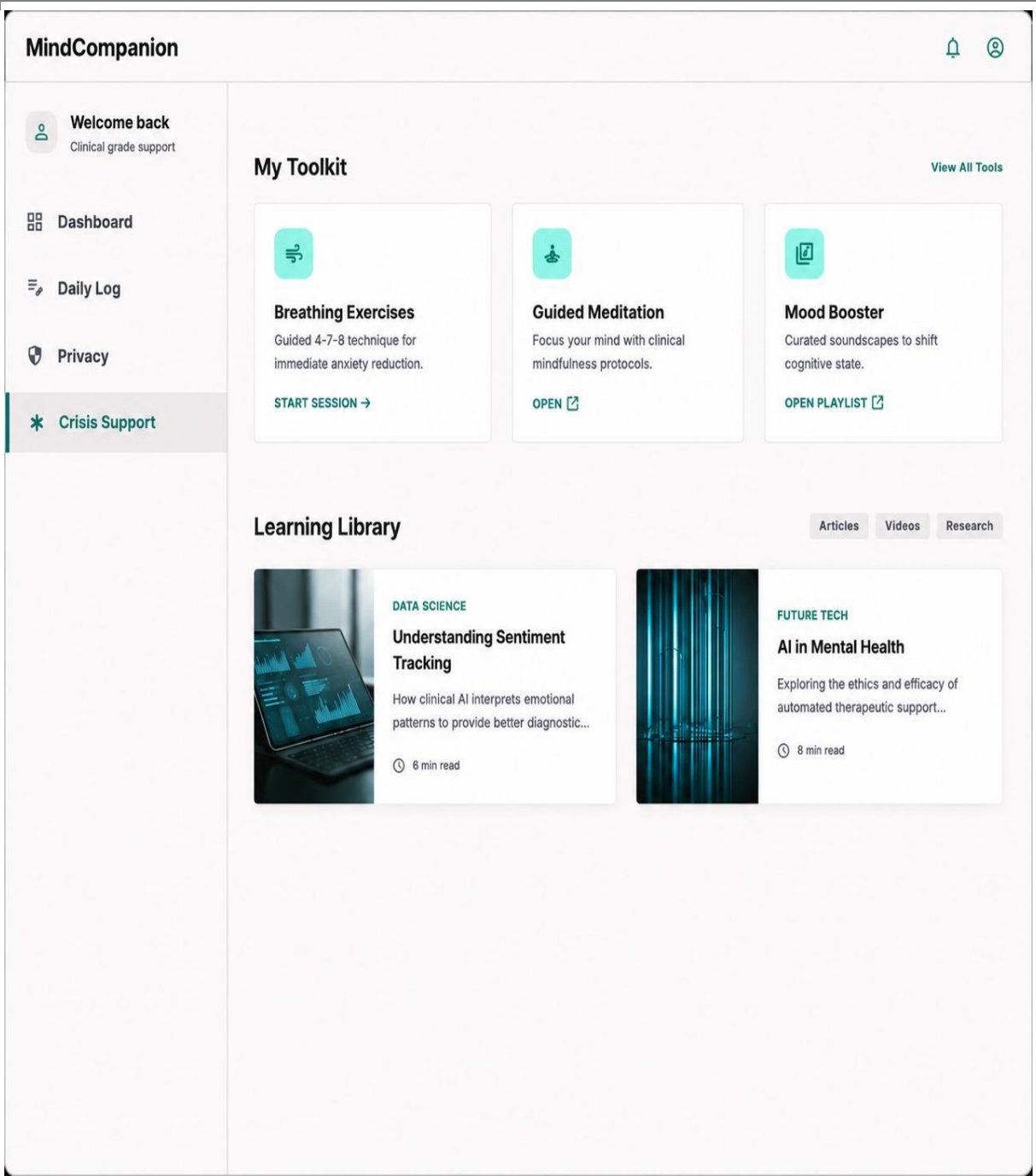


Figure: Screen Image — Crisis Support & Resources

8.2 Screen Objects and Actions

The table below lists the primary interactive objects on each screen and the action each triggers.

Screen	Object	Action
Dashboard	"START 30-MIN DETOX" button	Begins a guided digital-detox timer and logs the action against the current alert.
Dashboard	"DISMISS" button	Closes the current Content Sentiment Alert banner without starting a detox session.
Dashboard	Preventive Actions list (Mute Keywords, Enable Feed Filter, Schedule Nightly Pause)	Opens the corresponding settings panel for the selected preventive action.
Daily Log	Sentiment Volatility timeline bars	On hover/click, displays the date, sentiment score, and whether a risk flag was raised for that day.
Daily Log	Social Feed / Direct Messages toggle	Switches the Post & Comment Deep Dive panel between social-feed posts and direct-message content.
Sentiment Alert	"Take a 30-min break" button	Confirms the break action and closes the modal, updating the Alert status to Viewed.
Sentiment Alert	"Dismiss" button	Closes the modal without action and updates the Alert status to Dismissed.
Crisis Support	"START SESSION" (Breathing Exercises)	Launches the guided breathing exercise tool.
Crisis Support	"OPEN" (Guided Meditation) / "OPEN PLAYLIST" (Mood Booster)	Opens the corresponding external or embedded media resource.
Sidebar (all screens)	Dashboard / Daily Log / Privacy / Crisis Support links	Navigates to the corresponding screen within the application.

9. Appendix I

- How to design using UML (OOP): For guidance, please follow the instructions mentioned in the link: <http://agilemodeling.com/artifacts/>
- How and when to design ER diagrams: For guidance, please follow the instructions mentioned in the link: http://people.inf.elte.hu/nikovits/DB2/Ullman_The_Complete_Book.pdf
- Data flow diagrams: For guidance, please follow the instructions mentioned in the link and book: <http://www.agilemodeling.com/artifacts/dataFlowDiagram.htm> ; Software Engineering — A Practitioner's Approach by Roger Pressman.
- Architecture diagram: For guidance, please follow the instructions mentioned in the book: Ian Sommerville — Software Engineering, 9th Edition, Chapter 6.